NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
[KARACHI CAMPUS]

FAST School of Computing **Spring 2026**

EduSphere — Student Management System

Software Design & Analysis

Team Members:

Sana Munir Alam [24K-0573] |

Adeena Asif [24K-0628] |

Zaiyan Umer [24K-0771] |

Tushal Chhabria [24K-0835]

Submitted To: Mr. Syed Ahmed Khan

Table of Contents

Chapter 1: Introduction

1.1 System Overview

1.2 Problem Statement

1.3 Scope

1.4 Objectives

1.5 Users and Stakeholders

1.6 System Requirements Summary

1.7 Software Process Model

Chapter 2: UML Diagrams

2.1 Use Case Diagram

2.2 Class Diagram

2.3 Activity Diagram

2.4 Sequence Diagram

2.5 Component Diagram

2.6 Deployment Diagram

2.7 Timing Diagram

Chapter 5: Architectural View (4+1 Model)

Chapter 1: Introduction

1.1 System Overview

EduSphere is a comprehensive, web-based Student Management System (SMS) developed to digitize and centralize the academic and administrative operations of educational institutions. The platform serves as a unified ecosystem through which students, faculty members, and administrators interact with institutional data in a structured, role-based, and secure environment.

The system integrates the following core functional modules: user authentication and access control, student record management, course creation and registration, attendance tracking, grade management, feedback collection, and report generation. EduSphere replaces fragmented manual processes and isolated software tools with a coherent, scalable, and maintainable solution. Built on a three-tier architecture — a React-based frontend, a Java/Node.js backend API layer, and a relational database (MySQL or PostgreSQL) — the system is designed to be maintainable, extensible, and accessible across standard web browsers.

1.2 Problem Statement

Educational institutions continue to face significant operational challenges stemming from reliance on manual processes, spreadsheets, or isolated software tools for academic management. These challenges include:

- **Data redundancy and inconsistency** arise when student records, attendance logs, and grade entries are maintained across separate, unconnected systems or spreadsheet files. There is no single source of truth, making data reconciliation time-consuming and error-prone.
- **Delayed access to critical information** means that administrators, faculty, and students are unable to obtain timely data on academic performance, attendance patterns, or enrollment statuses. This delay limits early intervention in cases of academic difficulty or non-compliance.
- **Inefficient administrative workflows** caused by manual enrollment processing, grade submission, and report generation consume disproportionate staff time and introduce unnecessary bottlenecks in institutional operations.

- **Limited transparency and poor communication** between the student body, faculty, and administration further hinder effective academic governance, reducing accountability and trust within the institution.
- **A high risk of human error** is inherent in manual data handling, particularly for high-stakes records such as grades, attendance, and fee statuses, which can have serious academic and legal consequences.

EduSphere directly addresses each of these challenges by providing a centralized, automated, role-sensitive platform that ensures data integrity, workflow efficiency, and institutional transparency.

1.3 Scope

EduSphere is scoped to serve a single educational institution and encompasses the following functional boundaries:

In Scope:

- Role-based user registration and secure login (Student, Faculty, Admin)
- Student academic record management (personal details, academic history, enrolled courses)
- Course lifecycle management (creation, scheduling, allocation to faculty, and student registration)
- Digital attendance recording and monitoring per course section
- Grade entry, publication, and real-time GPA/CGPA calculation
- Institutional report generation (attendance summaries, grade reports, enrollment statistics)
- Course feedback submission by students
- System-generated notifications for enrollment, attendance alerts, and grade releases
- Administrative dashboards for oversight and data governance

Out of Scope:

- Financial management, fee collection, or accounts payable/receivable
- Integration with external Learning Management Systems (e.g., Moodle, Canvas)
- Mobile application development (the system is browser-accessible but not a native app)
- Alumni management and placement services
- Examination scheduling and invigilation management

1.4 Objectives

Primary Objectives:

The primary goal of EduSphere is to design and implement a well-structured, UML-modeled Student Management System that demonstrates clear separation of concerns across system modules. The system aims to produce a consistent and coherent software design supported by comprehensive UML artifacts.

Functional Objectives:

EduSphere aims to enable role-specific users to register and authenticate securely; manage and update student academic records; facilitate course creation, allocation, and registration; record and monitor attendance digitally; upload and access grades in real time; generate academic and administrative reports; and receive automated notifications regarding course activities, attendance thresholds, and grade publications.

Non-Functional Objectives:

The system shall maintain data security through hashed passwords and session management. It shall be responsive to concurrent user access without performance degradation. The architecture shall support future extensibility, including module additions without requiring system-wide refactoring.

1.5 Users and Stakeholders

- **Administrator:** Responsible for the overall management of the system, including user registration approvals, course creation, faculty allocation, and system-wide report generation. The administrator has the highest privilege level within the system.
- **Faculty Member:** Responsible for managing their assigned courses, recording attendance for enrolled students, uploading grades following assessment, and receiving course feedback from students. Faculty interact with the system primarily through the course and grade management modules.
- **Student:** Can register for available courses within permitted limits, view personal attendance records and grades, submit course feedback, and receive system notifications. Students interact with a read-and-request model for most academic data.
- **Authentication Service (External Actor):** Handles credential verification and identity token issuance for secure login, represented as an external service boundary in the system architecture.
- **Institutional Management (Indirect Stakeholder):** Benefits from the aggregated reporting and analytics outputs generated by EduSphere but does not directly interact with the system.

1.6 System Requirements Summary

Functional Requirements:

ID	Requirement
FR-01	The system shall allow users to register and log in using role-based credentials.
FR-02	Administrators shall be able to create, update, and deactivate student and faculty accounts.
FR-03	Faculty shall be able to create and manage courses, including scheduling and student capacity.
FR-04	Students shall be able to enroll in available courses subject to prerequisite and capacity constraints.
FR-05	Faculty shall be able to record and update attendance for each class session.
FR-06	Faculty shall be able to enter and publish grades per course assessment.
FR-07	The system shall automatically calculate and display GPA/CGPA upon grade publication.
FR-08	Students shall be able to submit anonymous or identified feedback on completed courses.
FR-09	The system shall generate reports on attendance, grades, and enrollment for administrators.
FR-10	The system shall send automated notifications upon enrollment confirmation, attendance warnings, and grade releases.

Non-Functional Requirements:

ID	Requirement
NFR-01	All user passwords shall be stored using a secure hashing algorithm (e.g., bcrypt).
NFR-02	The system shall maintain session security through token-based authentication (JWT).
NFR-03	The system shall support at least 500 concurrent users without degraded response time.
NFR-04	All web interfaces shall be responsive and accessible via standard browsers without plugin installation.
NFR-05	The database shall enforce referential integrity through foreign key constraints.
NFR-06	System modules shall be loosely coupled to allow independent deployment and testing.

1.7 Software Process Model

Adopted Model: Iterative and Incremental Development (aligned with the Unified Process)

EduSphere adopts an Iterative and Incremental process model, structured broadly along the four phases of the Unified Process (UP): Inception, Elaboration, Construction, and Transition.

Inception Phase involved stakeholder identification, high-level requirements gathering, and project scoping — producing the initial proposal and use case inventory.

Elaboration Phase focused on architectural risk resolution, UML modeling, and establishing the stable system baseline. All diagrams produced in this document correspond to this phase.

Construction Phase will involve incremental implementation of system modules — beginning with authentication and core user management, followed by course and enrollment functionality, then attendance and grading, and finally reporting and notifications.

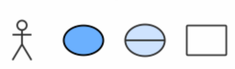
Transition Phase encompasses system testing, user acceptance testing (UAT), deployment, and handover documentation.

Rationale: The Iterative model is particularly well-suited to EduSphere for three reasons. First, the system is composed of clearly bounded, independently deliverable modules. Second, UML-based design artifacts allow requirements to be validated before full implementation. Third, the involvement of multiple user roles necessitates early prototyping and iterative feedback.

Edu Sphere (Use-Case)

Use case shape key
- Actor
- Use case
- Use case extension
- Untitled key item

Use case shape bank



USER

Login
Extension Points
Reset Password

<<include>>

Authenticate Credential

AUTHENTICATION SERVICE

Logout

<<extend>>

Reset Password

Get Profile

Update Profile

Student Dashboard

View Dashboard

Faculty Dashboard

Display

Admin Dashboard

<<include>>

STUDENT

Submit Course Feedback

Request Enrollment

<<include>>

Validate Seat Avaiability

Drop Course

<< include>>

Update Enrollment Status

View Attendance

View Grades
Extension Points
Download Grade Report
Request Grade Change

<< extend>>

Download Grade Report

<< extend>>

Request Grade Change

FACULTY

Assign Grade

<< include>>

Validate Enrollment Status

Mark Attendance

<< include>>

Update Grades

ADMIN

Approve Enrollment

<< include>>

<< include>>

Create Course

<< include>>

Validate Credit Hours

Assign Faculty to Course

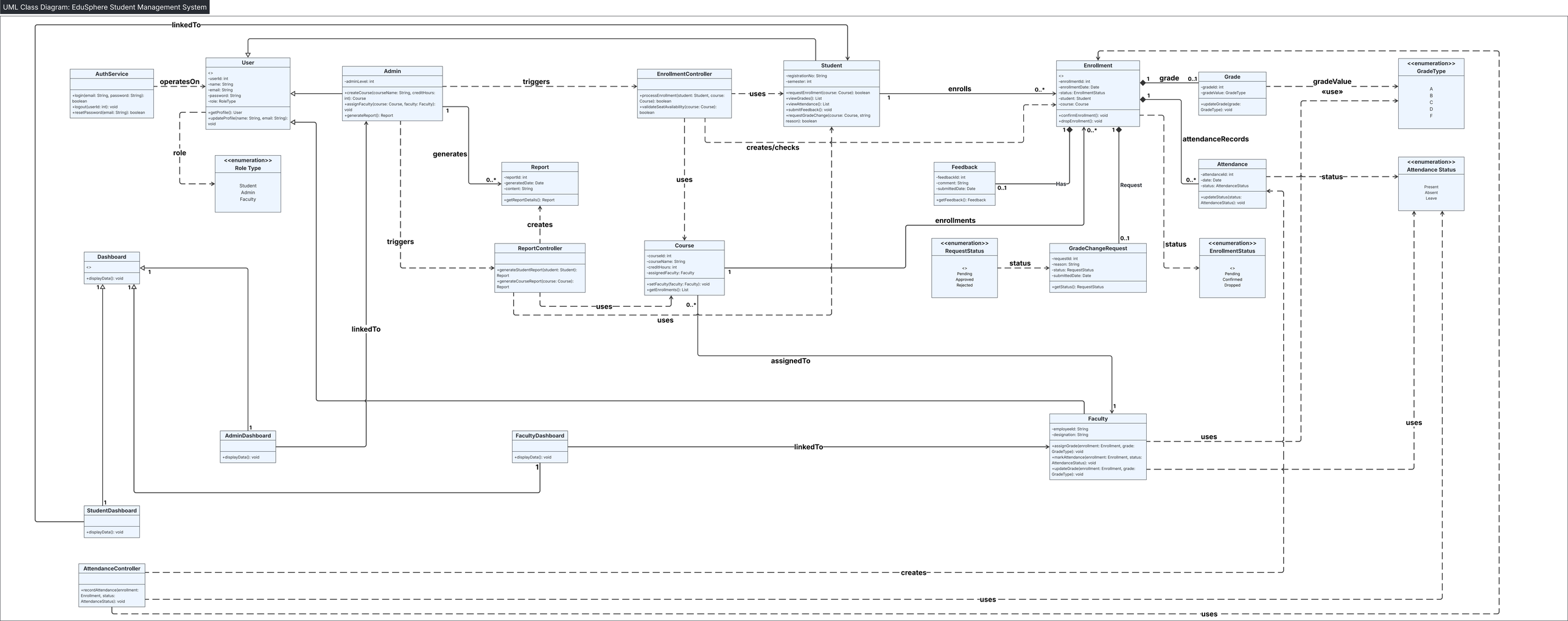
Generate Report

<< include>>

Get Report Details

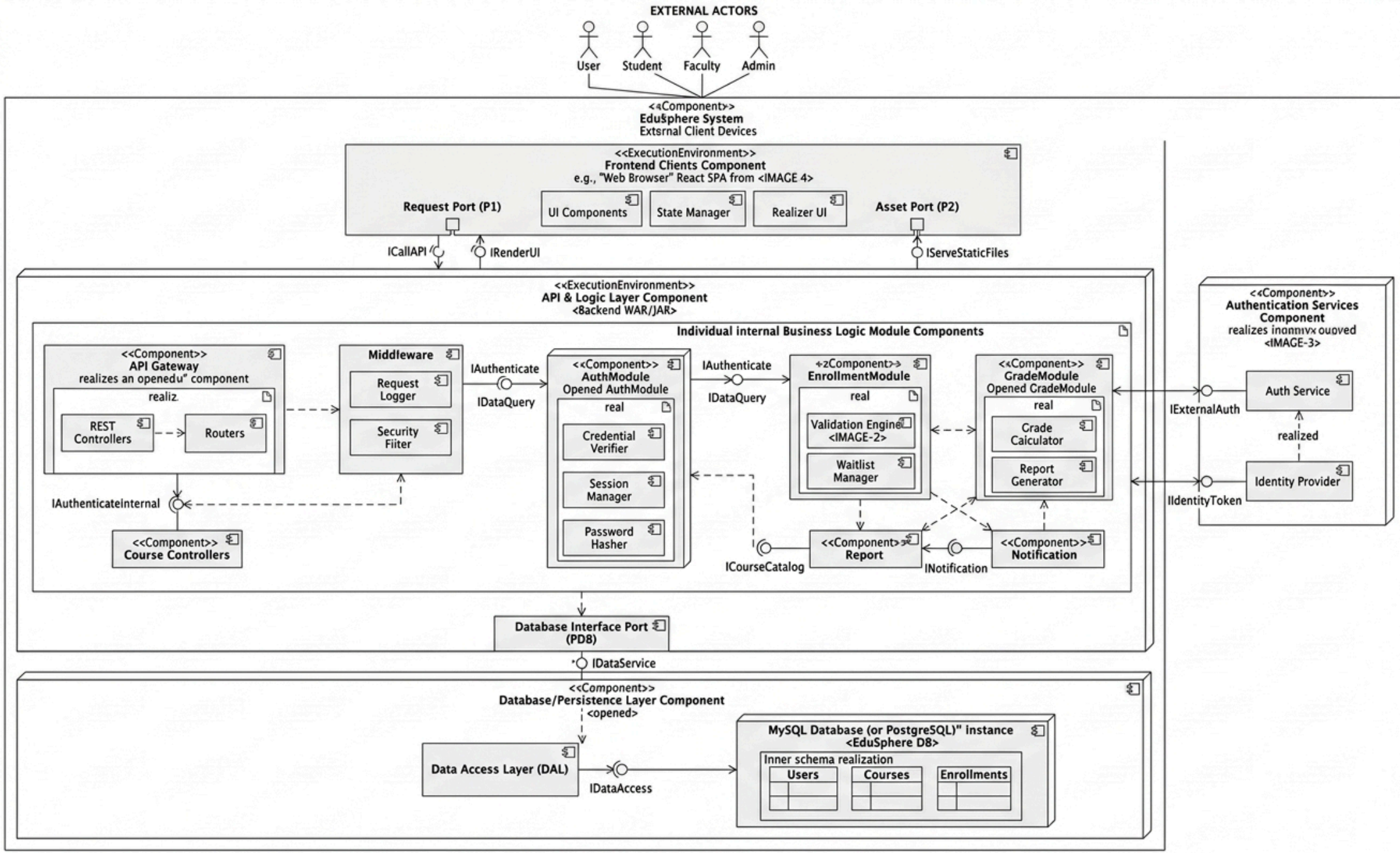
Student Report

Course Report

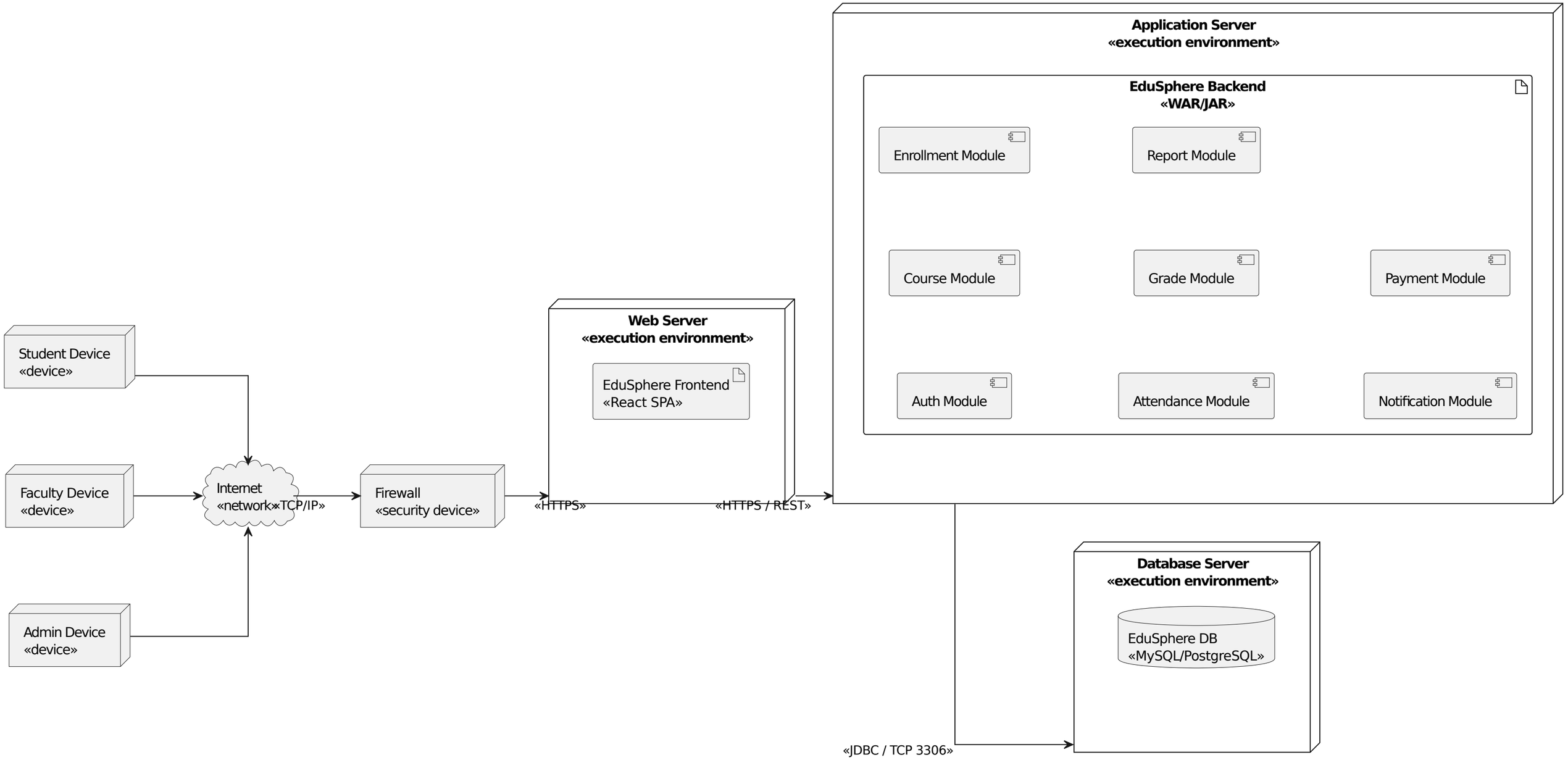

UML Class Diagram: EduSphere Student Management System
AuthService
+login(email: String, password: String): boolean
+logout(userId: int): void
+resetPassword(email: String): boolean
operatesOn
User
<>
-userId: int
-name: String
-email: String
-password: String
-role: RoleType
+getProfile(): User
+updateProfile(name: String, email: String): void
role
<<enumeration>>
Role Type
Student
Admin
Faculty
Admin
-adminLevel: int
+createCourse(courseName: String, creditHours: int): Course
+assignFaculty(course: Course, faculty: Faculty): void
+generateReport(): Report
triggers
generates
Report
-reportId: int
-generatedDate: Date
-content: String
+getReportDetails(): Report
creates
ReportController
+generateStudentReport(student: Student): Report
+generateCourseReport(course: Course): Report
triggers
uses
linkedTo
EnrollmentController
+processEnrollment(student: Student, course: Course): boolean
+validateSeatAvailability(course: Course): boolean
uses
uses
creates/checks
Course
-courseId: int
-courseName: String
-creditHours: int
-assignedFaculty: Faculty
+setFaculty(faculty: Faculty): void
+getEnrollments(): List
uses
assignedTo
Student
-registrationNo: String
-semester: int
+requestEnrollment(course: Course): boolean
+viewGrades(): List
+viewAttendance(): List
+submitFeedback(): void
+requestGradeChange(course: Course, string reason): boolean
enrolls
Enrollment
<>
-enrollmentId: int
-enrollmentDate: Date
-status: EnrollmentStatus
-student: Student
-course: Course
+confirmEnrollment(): void
+dropEnrollment(): void
enrollments
Feedback
-feedbackId: int
-comment: String
-submittedDate: Date
+getFeedback(): Feedback
Has
Request
<<enumeration>>
RequestStatus
<>
Pending
Approved
Rejected
status
GradeChangeRequest
-requestId: int
-reason: String
-status: RequestStatus
-submittedDate: Date
+getStatus(): RequestStatus
grade
Grade
-gradeId: int
-gradeValue: GradeType
+updateGrade(grade: GradeType): void
gradeValue
«use»
<<enumeration>>
GradeType
A
B
C
D
F
attendanceRecords
Attendance
-attendanceId: int
-date: Date
-status: AttendanceStatus
+updateStatus(status: AttendanceStatus): void
status
<<enumeration>>
Attendance Status
Present
Absent
Leave
status
<<enumeration>>
EnrollmentStatus
<>
Pending
Confirmed
Dropped
Dashboard
<>
+displayData(): void
linkedTo
AdminDashboard
+displayData(): void
FacultyDashboard
+displayData(): void
linkedTo
StudentDashboard
+displayData(): void
Faculty
-employeeId: String
-designation: String
+assignGrade(enrollment: Enrollment, grade: GradeType): void
+markAttendance(enrollment: Enrollment, status: AttendanceStatus): void
+updateGrade(enrollment: Enrollment, grade: GradeType): void
uses
uses
AttendanceController
+recordAttendance(enrollment: Enrollment, status: AttendanceStatus): void
creates
uses
uses

EduSphere - Complete UML Activity Diagram (Final Version)

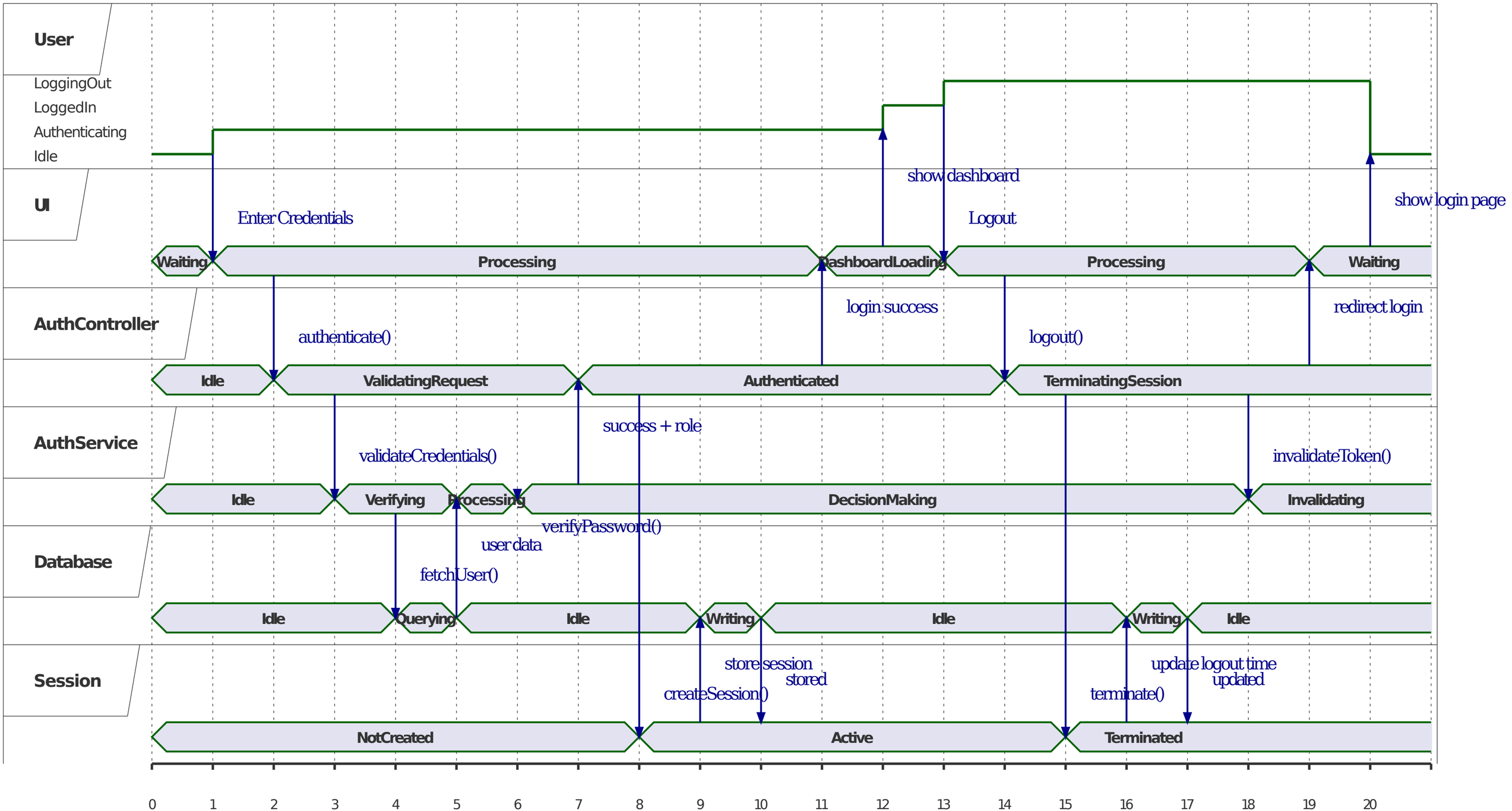
EduSphere - Final Comprehensive Sequence Diagram (Final Version)


EXTERNAL ACTORS
User
Student
Faculty
Admin
<<Component>>
EduSphere System
External Client Devices
<<ExecutionEnvironment>>
Frontend Clients Component
e.g., "Web Browser" React SPA from <IMAGE 4>
Request Port (P1)
UI Components
State Manager
Realizer UI
Asset Port (P2)
ICallAPI
IRenderUI
IServeStaticFiles
<<ExecutionEnvironment>>
API & Logic Layer Component
<Backend WAR/JAR>
Individual internal Business Logic Module Components
<<Component>>
API Gateway
realizes an openedu" component
realiz
REST Controllers
Routers
Middleware
Request Logger
Security Fiiter
IAuthenticate
IDataQuery
<<Component>>
AuthModule
Opened AuthModule
real
Credential Verifier
Session Manager
Password Hasher
IAuthenticate
IDataQuery
<<Component>>
EnrollmentModule
real
Validation Engine
<IMAGE-2>
Waitlist Manager
<<Component>>
GradeModule
Opened CradeModule
real
Grade Calculator
Report Generator
<<Component>>
Authentication Services Component
realizes inanniyx ouoved
<IMAGE-3>
Auth Service
realized
Identity Provider
IExternalAuth
IIdentityToken
IAuthenticateInternal
<<Component>>
Course Controllers
ICourseCatalog
<<Component>>
Report
INotification
<<Component>>
Notification
Database Interface Port
(PD8)
IDataService
<<Component>>
Database/Persistence Layer Component
<opened>
Data Access Layer (DAL)
IDataAccess
MySQL Database (or PostgreSQL)" instance
<EduSphere D8>
Inner schema realization
Users
Courses
Enrollments

EduSphere - Deployment Diagram



EduSphere - Timing Diagram (Authentication & Session Lifecycle)



Chapter 5: Architectural View (4+1 Model)

The 4+1 Architectural View Model, as proposed by Philippe Kruchten, provides five complementary perspectives on the EduSphere system architecture, each addressing the concerns of different stakeholders.

5.1 Logical View

The Logical View addresses the functional decomposition of EduSphere from the perspective of the end users, focusing on the major abstractions in the system.

The core logical structure revolves around the User hierarchy. The base User class is generalized into Student, Faculty, and Admin, each encapsulating role-specific behavior. The Course class serves as the central entity around which the majority of system activity is organized. The Enrollment association class links Students and Courses, while Attendance and Grade represent the two primary academic data artifacts that are generated throughout a course's lifecycle.

The logical packages include: auth (User, AuthController, SessionManager), courses (Course, EnrollmentController, EnrollmentDB), grades (Grade, GradeController, GradeCalculator), attendance (Attendance, AttendanceController), reports (ReportController, ReportGenerator), and notifications (NotificationService). Each package is cohesive and loosely coupled, adhering to the Single Responsibility Principle.

Key relationships in the logical view: Course aggregates Enrollment; Enrollment composes Grade; Student associates with Attendance; Faculty controls Course and Attendance operations. This structural arrangement ensures that data flows follow role-based access paths and that no single class accumulates disproportionate responsibility.

5.2 Process View

The Process View addresses the system's concurrency, thread management, and control flow, concerning developers and system integrators.

EduSphere's backend operates on a request-driven, multi-threaded model provided by the application server runtime. Each incoming HTTP request from the frontend spawns a thread (or is assigned a worker from a thread pool) that is responsible for the complete lifecycle of that request — routing through the API Gateway, processing in the relevant business logic module, database access via the DAL, and response construction.

Notable concurrent processes include the Enrollment Validation flow, where prerequisite checking and capacity verification are executed as parallel sub-processes (represented by fork-join constructs in the Activity Diagram), before a decision is made to confirm or reject enrollment. The Report Generation flow similarly fans out parallel queries to Attendance, Grade, and Enrollment databases and joins results before producing the final report artifact.

The Notification Service operates as a decoupled asynchronous process. When events such as grade publication or attendance threshold breaches occur, the notification dispatch is delegated to an asynchronous handler, ensuring the primary business transaction completes without being blocked by notification delivery latency. Session management is handled by the AuthModule's Session Manager, which issues JWT tokens upon successful authentication and validates them on every subsequent request.

5.3 Development View

The Development View addresses the organization of software modules and packages from the perspective of developers and maintainers.

EduSphere's codebase is organized following a modular, layered package structure:

- **The Frontend (React SPA)** is organized into feature-based modules: /auth, /student, /faculty, / admin, each containing their respective components, state management hooks, and API service calls. A shared /components library houses reusable UI elements.
- **The Backend** follows a standard MVC-adjacent package hierarchy: controllers (REST Controllers and Course Controllers), middleware (Request Logger, Security Filter), modules/auth (AuthModule with Credential Verifier, Session Manager, Password Hasher), modules/enrollment (EnrollmentModule with Validation Engine and Waitlist Manager), modules/grades (GradeModule with Grade Calculator and Report Generator), services/notification, services/report, and dal (Data Access Layer).
- **The Database schema** is organized into logically named tables: users, students, faculty, admins, courses, enrollments, attendance, grades, and feedback, with foreign key constraints enforcing relational integrity.

Build artifacts are modular: the backend compiles to a deployable WAR/JAR, and the frontend builds to a static bundle served from the Asset Port. This separation enables independent deployment and testing of frontend and backend components.

5.4 Physical View

The Physical View addresses the mapping of software artifacts to physical hardware nodes, concerning system engineers and deployment teams.

As described in the Deployment Diagram, EduSphere is deployed across four nodes:

- **The Client Node (User Device)** runs the React SPA in a standard web browser, communicating with the backend over HTTPS. No server-side rendering is required; all rendering is client-side.
- **The Application Server Node** hosts the backend API and Logic Layer (WAR/JAR). It serves as the sole entry point for business logic execution, handling authentication, enrollment, grading, attendance, and report generation. It communicates internally with the Database Server and externally with the Authentication Server.
- **The Database Server Node** hosts the relational database instance (MySQL or PostgreSQL — EduSphere DB). This node is isolated from direct client access, communicating only with the Application Server over the internal network, ensuring data security.
- **The External Authentication Server** hosts the Auth Service and Identity Provider, handling credential validation and token issuance. This separation allows the authentication mechanism to be upgraded, replaced, or scaled independently of the core application server.

For production deployment, the Application Server and Database Server should reside within a Virtual Private Cloud (VPC) or equivalent private network segment, with only the Application Server's HTTPS port exposed to the public internet. This physical topology reflects defense-in-depth security principles.

5.5 Use Case View (+1)

The Use Case View serves as the unifying perspective, cross-cutting all four architectural views by grounding them in the concrete scenarios that the system is designed to support. It is the "+1" in the 4+1 model because it provides the scenarios used to validate and illustrate the other four views.

The primary scenarios driving EduSphere's architecture are:

- **Login / Authentication (UC-01):** Drives the Logical View's User and AuthController classes, the Process View's session management thread, the Development View's /modules/auth package, and the Physical View's External Authentication Server node.
- **Course Enrollment (UC-03):** Drives the Logical View's Enrollment association class, the Process View's concurrent validation fork-join, the Development View's /modules/enrollment package, and the Physical View's Application-to-Database communication.
- **Grade Submission and Publication (UC-05):** Drives the GradeModule in all views, the asynchronous Notification Service in the Process View, and the Grade schema in the Database Server.
- **Report Generation (UC-09):** Drives the parallel data aggregation logic in the Process View, the Report Generator and DAL in the Development View, and the cross-table query patterns in the Physical View's Database Server.

Together, these scenarios confirm that every architectural decision — from class design to physical deployment — is traceable to a concrete user interaction, ensuring that EduSphere's architecture is not over-engineered and remains grounded in real stakeholder value.